# GPU-Accelerated Discrete Element Method (DEM) Simulation of a Snowball Avalanche Dynamics on NVIDIA Maxwell

## Attachment 1: Multi-GPU Extension Strategy

Maurizio Grisotto
*Politecnico di Torino*
Turin, Italy 2026
s336145@studenti.polito.it

***Abstract*—This attachment presents a theoretical analysis of extending the Angry Santa DEM snowball simulation to multiple GPUs. The analysis covers pipeline communication requirements, 1D domain decomposition along the slope axis, halo exchange sizing, a per-frame multi-stream pipeline, comparison with index-based partitioning, concrete code change maps, and an optional CUDA-aware MPI path for multi-node clusters.**

***Index Terms*—multi-GPU, domain decomposition, halo exchange, CUDA peer-to-peer, `cudaMemcpyPeerAsync`, MPI, DEM, particle simulation**

## I Pipeline Analysis

The current single-GPU pipeline executes five main stages per frame across two CUDA streams.

Inter-GPU communication is required at exactly two points per frame: the halo exchange before `neighborCollisionKernel`, and a small captured-mass reduce after `captureFromSnowpackKernel`. Both are cheap and maskable behind compute.

Each GPU will operate with 3 CUDA streams:

- `simStream`: main physics pipeline (existing)
- `gridStream`: grid construction (existing, concurrent with `forceTetherKernel`)
- `commStream` (*new CUDA stream*): pack halo → `cudaMemcpyPeerAsync` → unpack halo

The join before `neighborCollisionKernel` requires `simStream` to wait on both `evGridDone` (existing) and `evHaloDone` (new).

## II 1D Domain Decomposition

### A. Geometry

The ball rolls along the slope in the **s** direction (downhill). Shell particles cluster around the ball with radius $r_{\text{ball}} + r_{\text{tether}}$; the spatial distribution is essentially 1D along $s$. The proposed partitioning divides the $s$ axis into $G$ slices (one per GPU) with boundaries $s_0 < s_1 < \cdots < s_G$. GPU $g$ owns all particles with $s_g \leq \text{posS} < s_{g+1}$. The halo region for GPU $g$ consists of particles in the adjacent slices $[s_{g-1} - h, s_g)$ and $[s_{g+1}, s_{g+1} + h)$, where $h$ is the halo width (Section II-B).

### B. Halo Exchange

The halo width must cover the maximum interaction radius of `neighborCollisionKernel`. The effective cutoff is $\max(\texttt{cohesionRadius}, 2 \cdot \texttt{particleRadius})$, so the required halo width is:

$$h = \max\big(\texttt{cellSize}, \texttt{cohesionRadius}\big)$$

The halo width must be computed at runtime from the actual parameter values, not hardcoded to `cellSize`.

### C. Snowpack Replication and Ball Broadcast

The snowpack is replicated on each GPU; the only post-capture communication is reducing `h_captureCount` and `h_captureMass` across GPUs. The `SnowballState` struct ($\approx$ 60 B) is broadcast to all GPUs via `cudaMemcpyAsync` (host→device) at the start of each frame at negligible cost.

## III Comparison with Index-Based Partitioning

The simpler alternative is to assign to each GPU $g$ the particles with index $i \bmod G = g$ (round-robin), or contiguous blocks $[gN/G, (g+1)N/G)$. With index-based partitioning, particles on the same GPU are not spatially localised: a particle's neighbors in the hash grid can reside on any other GPU. This has two severe consequences:

- `O(N)` communication: each GPU must send/receive all positions and velocities to/from every other GPU, because it cannot know in advance which particles will be neighbors in the spatial grid.
- Duplicated grid: each GPU must build a grid over all particles (not only its own), because the neighbor search requires global positions. This eliminates the advantage of partitioning the grid build.

Index-based partitioning requires each GPU to exchange *all* positions with every other GPU and to build a grid over *all* particles. Domain decomposition exploits the strong spatial coherence of shell particles around the ball.

TABLE I
INTER-GPU COMMUNICATION PER PIPELINE STAGE

| # | Kernel / Stage | Communication |
|---|---|---|
| 1 | `captureFromSnowpackKernel` | **Partial.** Snowpack is read-only; replicated on each GPU (∼2.7 MB for 100 k particles: 6 float + 1 int = 28 B). Captured mass must be *reduced* across all GPUs before `updateCoreMassAfterCapture`. |
| 2 | `forceTetherKernel` | **None.** Per-particle: gravity, damping, tether. |
| 3 | Grid build (hash + sort) | **None.** Each GPU builds its own local grid. |
| 4 | `neighborCollisionKernel` | **Yes.** Particles near subdomain boundaries require *halo* data from the adjacent GPU (Section II-B). |
| 5 | `integrateGroundKernel` | **None.** Per-particle semi-implicit Euler integration. |

# IV Code Change Map

## A. New Data Structure

```
struct PerGPUData {
    int            deviceId;
    ParticleSystem shell;      // N/G particles + halo space
    GridData       grid;       // sized for local + halo
    Snowpack       snowpack;   // replicated read-only copy
    cudaStream_t   simStream, gridStream;
    cudaStream_t   commStream;  // NEW
    cudaEvent_t    evFork, evGridDone;
    cudaEvent_t    evHaloDone;  // NEW
    // Halo buffers: pos(xyz) + vel(xyz) + mass + radius +
        state
    char  *haloSendBuf, *haloRecvBuf;
    int    haloSendCount, haloRecvCount;
    // Capture accumulators (pinned host + device)
    int   *d_captureCount; float *d_captureMass;
    int   *h_captureCount; float *h_captureMass;
};
```

Listing 1. Per-GPU state struct

## B. New Kernels

- `packHaloKernel`: scans particles with $|\text{posS} - s_{\text{border}}| < h$ and packs them into a contiguous buffer (8 float + 1 int per particle).
- `unpackHaloKernel`: appends received halo particles at indices $[\texttt{shellN\_local}, \texttt{shellN\_local} + \texttt{haloRecvCount})$ of the local `ParticleSystem`, so `neighborCollisionKernel` finds them in the spatial grid.

## C. Grid Sizing for Halo Particles

```
// allocateGrid signature: (GridData&, int capacity, const
    SimParams&)
int maxHaloSize = (int)(shellLocalCap * 0.10f); // 10%
    headroom
allocateGrid(grid, shellLocalCap + maxHaloSize, params);

// Halo buffer size per border:
size_t haloBytes = maxHaloParticles *
    (8 * sizeof(float) + sizeof(int));
// 8 floats: posX/Y/Z, velX/Y/Z, mass, radius
// 1 int:    state  (to skip INACTIVE ghosts in collision
    kernel)
// attachLocalX/Y/Z NOT included: halo particles are read-
    only ghosts
```

## D. Particle Migration

Occasionally a particle crosses the subdomain boundary (tethered to the ball as it moves). Migration must transfer **all 16 SoA arrays** of `ParticleSystem`:

```
posX, posY, posZ,             // 3 float
velX, velY, velZ,             // 3 float
forceX, forceY, forceZ,       // 3 float
mass, radius, wetness,        // 3 float
state,                        // 1 ShellParticleState (int)
attachLocalX, attachLocalY, attachLocalZ  // 3 float --
    CRITICAL
```

The `attachLocal*` arrays hold body-frame tether anchors computed once at capture via `quatInvRotateVec`. They are **not reconstructible** from world-space state; omitting them from migration would cause tether forces to point in the wrong direction.

## E. Modifications to `copyParamsToGPU`

```
// Must be called once per device at startup
for (int g = 0; g < numGPUs; g++) {
    cudaSetDevice(g);
    copyParamsToGPU(params); // writes d_params
        __constant__ on device g
}
```

## F. P2P Access Initialization

`cudaDeviceEnablePeerAccess` operates from the *current* device context; both directions must be enabled from the correct device:

```
for (int g = 0; g < numGPUs - 1; g++) {
    cudaSetDevice(g);
    cudaDeviceEnablePeerAccess(g + 1, 0); // g -> g+1
    cudaSetDevice(g + 1);
    cudaDeviceEnablePeerAccess(g, 0);     // g+1 -> g
}
```

If P2P is unavailable (PCIe without IOMMU bridge), the fallback is to transit through a pinned host staging buffer (2× latency, functionally identical).

## G. Small-N Fused Path

The single-GPU code switches to `shellPhysicsFusedKernel` when $\texttt{shellN} \leq 2048$. In multi-GPU, the per-GPU count $\texttt{shellN}/G$ may fall below this threshold even at moderate total $N$. The dispatcher must check the **local** particle count on each GPU and select the fused path independently per device.

# V CUDA-Aware MPI for Clusters

## A. Motivation

The peer-to-peer mechanism described in Section IV (`cudaMemcpyPeerAsync`) only works between GPUs connected to the *same* CUDA driver instance, i.e. on a single physical node. Scaling to a multi-node cluster (16, 32, 64+ GPUs) requires **MPI** (Message Passing Interface) to transfer data across the network.

## B. Classic MPI vs. CUDA-Aware MPI

With *classic* MPI, transferring a GPU buffer to another node requires four explicit steps:

1) `cudaMemcpy` D→H: copy halo from GPU to host pinned buffer;
2) `MPI_Isend`: send host buffer over the network;
3) `MPI_Irecv`: receive into host buffer on the remote node;
4) `cudaMemcpy` H→D: copy received data from host to GPU.

Steps 1 and 4 add two full PCIe traversals per transfer, doubling effective latency and consuming host CPU time for staging.

*CUDA-aware MPI* (e.g. OpenMPI $\geq$1.7 with `--with-cuda`, or MVAPICH2-GDR) recognises device pointers passed directly to `MPI_Isend`/`MPI_Irecv` and, when **GPUDirect RDMA** is available (NVLink or RDMA-capable NIC such as Mellanox InfiniBand), it transfers data directly from GPU to network card without involving the host CPU or host RAM at all.

TABLE II
CLASSIC MPI VS. CUDA-AWARE MPI FOR HALO EXCHANGE

| | Classic MPI | CUDA-Aware MPI |
|---|---|---|
| Data path | GPU→CPU→NIC→CPU→GPU | GPU→NIC→GPU |
| Extra copies | 2 (D→H + H→D) | 0 |
| Host staging buffer | required | not required |
| Latency | ∼2× higher | baseline |
| Code change | host copy + MPI call | MPI call only |

## C. Implementation

With CUDA-aware MPI, device pointers are passed directly to the MPI primitives. The halo pack/unpack kernels are unchanged; only the transfer call differs:

```
launchPackHalo(gpuData, commStream); // fill haloSendBuf on GPU

// Device pointers passed directly -- no host staging copy
MPI_Isend(gpuData.haloSendBuf, haloBytes, MPI_BYTE,
          neighborRank, tag, MPI_COMM_WORLD, &sendReq);
MPI_Irecv(gpuData.haloRecvBuf, haloBytes, MPI_BYTE,
          neighborRank, tag, MPI_COMM_WORLD, &recvReq);

// Overlap network transfer with forceTetherKernel on simStream,
// then wait before neighborCollisionKernel
MPI_Waitall(2, reqs, statuses);
launchUnpackHalo(gpuData, commStream);
```

Listing 2. Halo exchange with CUDA-aware MPI (non-blocking)

The captured-mass reduce across all nodes maps to a single MPI collective, avoiding any manual reduction loop:

```
// Each rank holds its local accumulated mass after captureKernel
MPI_Allreduce(&localCaptureMass, &totalCaptureMass,
              1, MPI_FLOAT, MPI_SUM, MPI_COMM_WORLD);
// totalCaptureMass now available on all ranks for ball update
updateCoreMassAfterCapture(ball, totalCaptureCount,
                           totalCaptureMass, snowDensity);
```

Listing 3. Captured-mass reduce with MPI

## D. Deployment Considerations

If GPUDirect RDMA is unavailable, CUDA-aware MPI falls back transparently to the classic D→H→network→H→D path, so the code remains correct and portable across all cluster configurations.

# VI Conclusions

1) **Highly favorable for multi-GPU**: `neighborCollisionKernel` is both the heaviest stage and the only one requiring significant communication, giving a comm/compute ratio of $< 0.1\%$.
2) **1D domain decomposition along** $s$ is $\sim$25$\times$ more communication-efficient than index-based partitioning, exploiting the strong spatial coherence of shell particles around the ball.
3) **Halo width** must be $\max(\texttt{cellSize}, \texttt{cohesionRadius})$, computed at runtime to remain correct under all CLI configurations.
4) **Migration must transfer all 16 SoA arrays**, especially `attachLocal*` (body-frame tether anchors, computed once at capture and not reconstructible from world-space state).
5) **Expected scaling**: nearly linear up to 4–8 GPUs; $\sim$3.8$\times$ speedup at 4 GPUs with $N = 200\,\text{k}$ active shell particles.
6) **Small-N fused path**: the `shellPhysicsFusedKernel` threshold must be evaluated independently per device in multi-GPU, since the local $N/G$ may drop below 2048 even when total $N$ is large.
7) **Minimal code impact**: changes are confined to `main.cu` (multi-device loop + capture reduce), two new lightweight kernels (`packHalo`, `unpackHalo`), one additional stream per GPU, and grid-sizing adjustments. The existing physics kernels require **no modification**.